

Trion

A Two-System Architecture for Vision-Language-Action Driving

Design proposal: systems, messages and use cases

Draft for stakeholder review · 06 September 2026

Scope: this document proposes how the driving stack is split into two systems and what passes between them. It deliberately does not design the models inside either system - that work should start only once the interface here has buy-in.

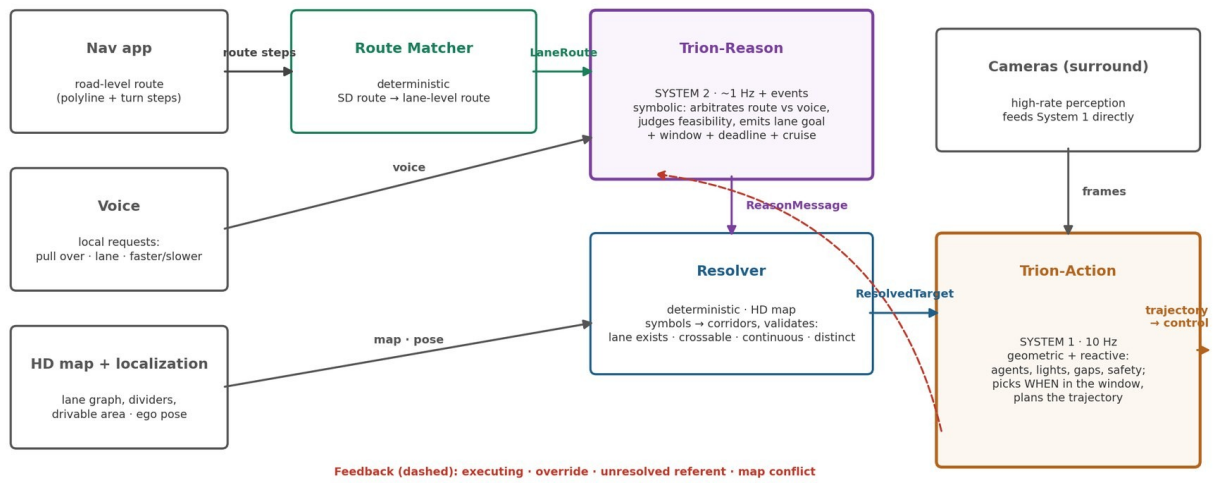
1. Executive summary

We propose splitting the driving model into two systems that run at different rates and talk through a small, symbolic, inspectable interface:

- **Trion-Reason (System 2, ~1 Hz).** Decides vehicle behaviour that tolerates a slow reaction: which lane to be in for the route, how fast to cruise relative to the limit, where to pull over, and how to reconcile a spoken request with the navigation route. It emits symbols - a lane-relative goal with a window and a deadline - never geometry.
- **Trion-Action (System 1, 10 Hz).** Does everything that must react within a second: other agents, traffic lights, gap acceptance, safety. It receives a resolved, map-anchored target and produces the trajectory the vehicle drives.
- **A deterministic Resolver in between.** Turns the symbols into lane corridors from the HD map and validates them - the lane exists, the marking is crossable, the target continues, it is a distinct lane. Requests that fail are rejected with a reason, not driven.
- **A deterministic Route Matcher in front.** Turns a road-level route from a navigation app ('turn right in 180 m') into lane-level facts: which lanes make the turn, how many lane changes that is from here, how far away.

The boundary between the two systems is drawn by time constant, not by topic: anything that can change in under a second belongs to Trion-Action; anything measured in seconds to minutes belongs to Trion-Reason. Lane change is split exactly there - whether and roughly where is slow; when, the gap, is fast.

A working mock of the whole chain exists today. No neural network runs in it: the two systems are rule-based stand-ins, while the Resolver and Route Matcher are real HD-map logic on nuScenes. It exercises the interface end to end on four scenes and is what the figures in this document show. What we are asking for is agreement on the split and the messages, so that model work on each side can proceed against a fixed contract.



Slow, symbolic decisions (route, lane, preference, pull-over) on top; fast, geometric reaction (agents, lights, gaps) on the bottom. The boundary is time constant, not topic.

Figure 1. The two systems and the deterministic modules between them. Solid arrows are messages; the dashed arrow is the feedback channel.

2. Why two systems

2.1 Where a single end-to-end model runs out of room

We evaluated Qwen-Drive-1.0, a strong single-model baseline (a 4B vision-language model with a planning head reading its internal state), on real data. Four limits showed up that are properties of the single-model shape, not of that model in particular:

- **The command interface is baked into the weights.** Navigation reaches the planner as one of three classes - straight, left, right - encoded as a one-hot into small networks and as text in the prompt. There is no vocabulary for 'change lane', 'pull over' or 'be in the right lane before the exit'. Adding one means retraining the planner.
- **One model cannot be both fast and deliberative.** The vision-language backbone that reasons about a scene is too heavy to run at 10 Hz on surround cameras; the planner that must run at 10 Hz is chained to it because it reads the backbone's state directly.
- **The route is not an input at all.** The model drives to a discrete command, not to a destination. A navigation app's route has nowhere to go.
- **Behaviour is hard to inspect or test.** When the plan is wrong there is no intermediate artefact that says whether the decision was wrong or the execution was. Open-loop displacement error also hides steering quality: on one scene the wrong turn command scored better than the recorded one because longitudinal error dominated the metric.

2.2 The design logic

Driving decisions differ enormously in how fast the thing being reacted to can change. A pedestrian, a closing gap or a traffic light changes in well under a second. A route, the lane needed for it, a speed preference, a construction zone or a request to pull over changes over seconds to minutes. Building one model for both forces it to run at the fast rate with the slow system's cost, or to carry the slow system's reasoning at a rate where it cannot afford it.

Splitting on that axis gives each side what it needs:

	Trion-Reason (System 2)	Trion-Action (System 1)
Time constant	seconds to minutes	under a second
Rate	~1 Hz, plus events (voice, reroute)	10 Hz
Owns	route following, lane choice, cruise preference, pull-over, voice arbitration, which traffic	agents, gaps, traffic-light state, following, yielding, merging, emergency braking,

	light applies	trajectory
Output	symbols: lane goal + window + deadline, cruise, planned stop	geometry: the trajectory the controller drives
Inputs	route (lane-level), voice, low-rate scene context, HD-map state	surround cameras at high rate, the resolved target
Data regime	sparse, language- and map-rich	dense, interaction-rich; ultimately closed-loop
Testable against	derived route/lane labels, preference adherence	closed-loop safety and comfort given ground-truth targets

Three properties follow from the split and are the reason to want it:

- **Interpretability.** Every decision Trion-Reason makes is a few readable symbols with a reason attached. A lane change that went wrong can be traced to 'requested RIGHT with a 45 m window' or to the execution, never to an opaque blob.
- **Independent evaluation.** Each system is measured against the interface: Trion-Reason against derived lane and speed targets, Trion-Action against closed-loop outcomes given correct targets. A hierarchy that cannot be evaluated per half is not worth having.
- **Independent development.** The two sides have different data, different sizes, different latency budgets and different teams. A fixed contract lets them move separately.

2.3 What Trion-Reason must not do

Because it is slow, Trion-Reason handles no interactive scenario. It never says 'follow that car' or 'yield to that pedestrian'; it says 'be in the right lane before the exit at 400 m' and Trion-Action handles every car in between. The one temptation to resist is letting 'slow-tolerant' drift toward 'anything symbolic': a stop sign is static, but the pedestrian stepping off the kerb at it is not. Trion-Reason may plan the stop; Trion-Action owns the pedestrian.

3. The components

3.1 Route Matcher (deterministic)

A navigation app hands over a road-level route: a polyline at GPS accuracy and turn steps such as 'turn right onto X in 180 m'. It carries no lane information. The Route Matcher bridges to the HD map: it finds the same-direction lanes across the road at the ego, walks each forward to the next junction with a real turn, and reports which lanes make the requested turn, how many lane changes that is from the ego's lane, and the distance. This is the LaneRoute message.

3.2 Trion-Reason (System 2)

Reads the LaneRoute, the voice request, the speed preference and low-rate scene context. It does two things a symbolic reasoner is for:

- **Feasibility.** N lane changes need roughly $N \times 2.5$ s of travel plus a margin. If the junction is closer than that, the answer is 'continue and ask the nav app to reroute', never 'force the gap'.
- **Arbitration.** The route is the default. A voice request is a scoped override: it wins until done, then the route resumes, and the message records which one asked. A two-change manoeuvre is emitted as a first goal plus an explicit pending follow-up.

Its output is the ReasonMessage: a lane-relative goal (KEEP, LEFT, RIGHT, PULL_OVER) with a window and a deadline in metres, a cruise preference relative to the speed limit, a planned stop at a static map element, which traffic light applies, and a reason.

3.3 Resolver (deterministic)

Turns symbols into geometry from the HD map and validates them. It localizes the ego in a lane, builds the current corridor by walking lane connectivity, classifies what lies beside the road every few metres - a lane, a junction, nothing - and fits the requested window to the stretch where a proper lane exists. It then checks that the marking is crossable, that the target lane continues past the window, and that it is a distinct lane rather than an overlapping map record. Any failure rejects the request to KEEP with the reason. Its output is the ResolvedTarget: both corridors, the fitted window, a speed cap and any stop point.

The Resolver is the component that makes the symbolic interface safe. Trion-Reason may ask for a lane that does not exist; the Resolver says so, and nothing is driven.

3.4 Trion-Action (System 1)

Receives the ResolvedTarget and the surround cameras. It owns all reactivity - agents, gaps, lights, safety - and chooses when inside the window to make the move. Its output is the trajectory: 50 points at 10 Hz over 5 s, each (x, y, heading), the same shape as today's planner. Precedence is explicit: it may always reduce speed below the cap and may shift laterally within the corridor's tolerance; it never changes lane against the target and never exceeds the cruise cap. Emergency actions are allowed and reported.

3.5 Feedback channel

Trion-Action reports each tick: which target it is executing, any override (speed reduced, emergency brake, path deviation), any referent it could not resolve, and whether perception disagrees with the map. Trion-Reason treats these as events. Without this channel a hierarchy cannot be audited: it is what tells you afterwards whether a bad outcome was the decision or the execution.

4. The messages

Four messages carry the whole design. They are small, typed, timestamped and latched; the transport is unimportant, the semantics are not.

4.1 LaneRoute (Route Matcher → Trion-Reason)

field	meaning
manoeuvre	CONTINUE · TURN LEFT · TURN RIGHT · ARRIVE (from the nav step)
at_m	distance to the junction where it happens, from HD geometry
lanes, current_lane	how many same-direction lanes here, and which the ego is in
valid	the lanes that lead into the requested turn
changes_needed, direction	lane changes from the ego's lane to the nearest valid one
matched, reason	false when the route cannot be tied to the map, with why

4.2 ReasonMessage (Trion-Reason → Resolver)

field	meaning
lateral	KEEP · LEFT · RIGHT · PULL_OVER - relative to the ego's lane, never geometry
window_m	complete the manoeuvre between these distances; Trion-Action picks the moment
deadline_m	after this the goal is no longer wanted; urgency for gap acceptance rises toward it
cruise	speed preference in notches relative to the speed limit
planned_stop	a stop at a static map element (pull-over point, destination), or none
which_light	which traffic light applies to this route through the junction
source	ROUTE or VOICE - who asked, for logs and for arbitration audits
pending	the next goal in a sequence, e.g. '+1 RIGHT before the junction'
reroute	the route's manoeuvre cannot be made; the nav app should reroute
why	a sentence of reasoning, for logs and for the driver

4.3 ResolvedTarget (Resolver → Trion-Action)

field	meaning
lateral, requested, fallback	what will be driven, what was asked, whether

	it was rejected
current_corridor, target_corridor	lane centrelines with lateral tolerance, map-anchored
window_m, deadline_m	the window after fitting to the map, and the clipped deadline
speed_cap	speed limit adjusted by the cruise preference
stop_x	where to come to rest, if a stop is planned
checks	every validation with pass/fail and a note - also what goes back on Feedback

4.4 Feedback (Trion-Action → Trion-Reason)

field	meaning
target_seq, executing	which ResolvedTarget is being driven, and whether it is
override	NONE · SPEED_REDUCED · EMERGENCY_BRAKE · PATH_DEVIATION
unresolved_ref	a referent perception could not find
map_conflict	perceived lane disagrees with the map beyond tolerance

4.5 Frames, timing and failure

- **Map-anchored, not ego-anchored.** At 15 m/s the car moves 15 m between Trion-Reason ticks. Targets expressed as lanes stay valid; Trion-Action re-transforms them each cycle.
- **Latched with expiry.** Trion-Action holds the last valid target. If none arrives for tens of seconds it falls back to lane-keep at the current speed and raises a flag.
- **Blend on change.** A new target cross-fades from the old over about half a second so a changed mind is not a steering jerk.
- **Rejection is an outcome.** A failed Resolver check keeps the previous target and reports the rejection; nothing half-resolved is ever driven.
- **Localization loss.** Below a confidence threshold the Resolver emits a perception-only lane-keep and Trion-Action drives what it sees.

5. Use cases

Each case is stated as inputs, what each component does, and the outcome. The first seven run in the mock today on nuScenes scenes; the figures are its output. Two conventions: distances are ahead of the ego along the road, and the mock assumes an urban limit of 50 km/h because the nuScenes map carries none.

UC1 · Route-driven single lane change (scene 132)

Inputs: nav step 'turn right at the next junction'; voice 'keep going'; ego 12 m/s in the left of two lanes.

- **Route Matcher:** right turn at 103 m; valid lane is lane 2; one change right.
- **Trion-Reason:** 103 m is enough for one change at 12 m/s → RIGHT, window 10-50 m, deadline 88 m, source ROUTE.
- **Resolver:** the right side is a junction for the first 60 m; the window is fitted to 64-88 m where a proper lane exists; marking DOUBLE_DASHED_WHITE; target continuous and distinct → accepted.
- **Trion-Action:** this tick's 5 s trajectory is the approach; the change is scheduled at 72 m and executes on a later tick.

Outcome: the vehicle will be in the turn lane well before the deadline, and the map - not the reasoner - decided where the change can happen.



Figure 2. UC1 in the mock: the fitted window sits past the roadside junction.

UC2 · Two-change sequence with a deadline (scene 189)

Inputs: nav step 'turn right at the next junction'; voice 'keep going'; ego 12.5 m/s in the kerb lane of three.

- **Route Matcher:** right turn at 127 m; valid lane is lane 3; two changes right.

- **Trion-Reason:** two changes fit in 127 m → RIGHT now, window 10-51 m, deadline 71 m, pending '+1 RIGHT before the junction'.
- **Resolver:** window fitted to 20-51 m past a short junction at 8-16 m; all checks pass.
- **Trion-Action:** commits at 31 m, ends 3.3 m to the right in the middle lane at 13.9 m/s; the second change follows once Feedback reports the first complete.

Outcome: a multi-step manoeuvre expressed as an explicit sequence, each step validated separately.

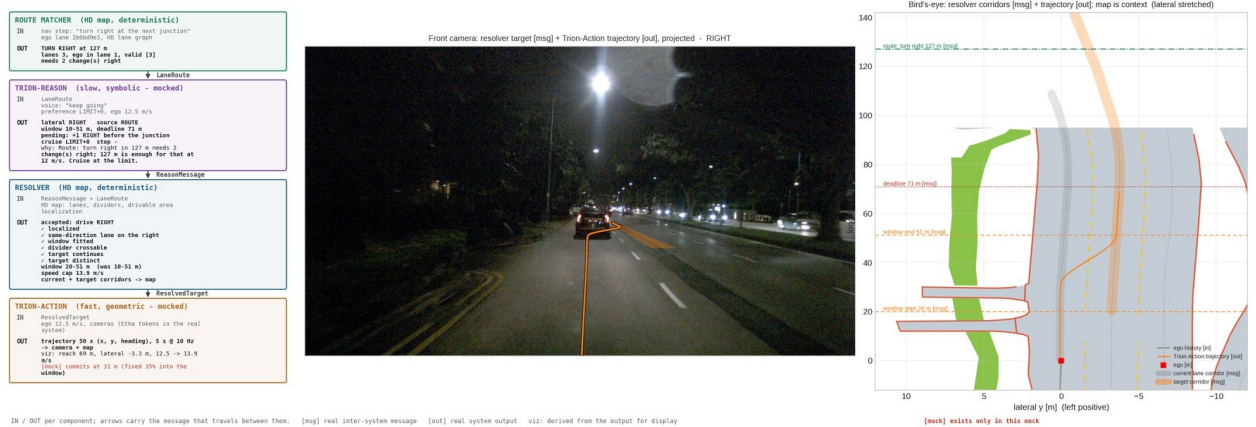


Figure 3. UC2: the first of two changes, with the pending second one on the reason card.

UC3 · Turn too close: continue and reroute (scenes 203, 147)

Inputs: nav step 'turn right at the next junction'; ego 15 m/s in the kerb lane; the turn is 33 m away and needs two changes.

- **Route Matcher:** right turn at 33 m; two changes right.
- **Trion-Reason:** two changes need about 110 m at this speed; 33 m is not safely possible → KEEP, reroute requested, with the reason.
- **Resolver / Trion-Action:** hold the lane at the cruise cap.

Outcome: the same behaviour a human driver shows when an exit comes too late: miss it and take the next route. Deadline pressure never becomes 'force the gap'.

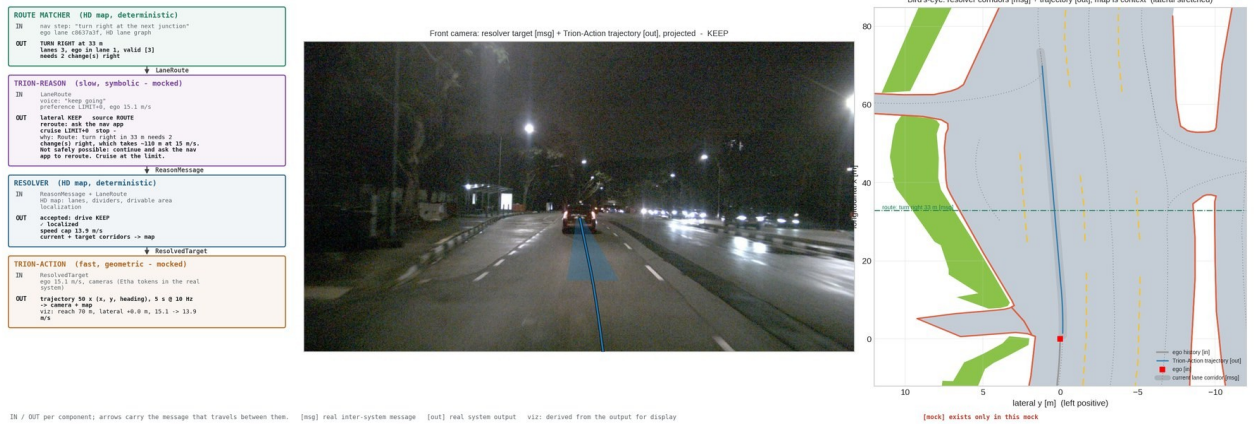


Figure 4. UC3: an honest refusal with a reroute request, not a lunge.

UC4 · Voice overrides the route (scene 132 + 'pull over')

Inputs: nav step 'turn right at the next junction'; voice 'pull over'.

- **Trion-Reason:** voice is a scoped override → PULL_OVER, source VOICE, window 10-40 m; pending 'route: RIGHT for the turn'; the message says the route resumes after and the nav app reroutes if the turn is missed.
- **Resolver:** kerb-side boundary 2.2 m left → a virtual pull-over lane; stop at 28 m.
- **Trion-Action:** eases 1.1 m to the kerb and comes to rest at 0 m/s.

Outcome: a spoken request wins locally without deleting the route.

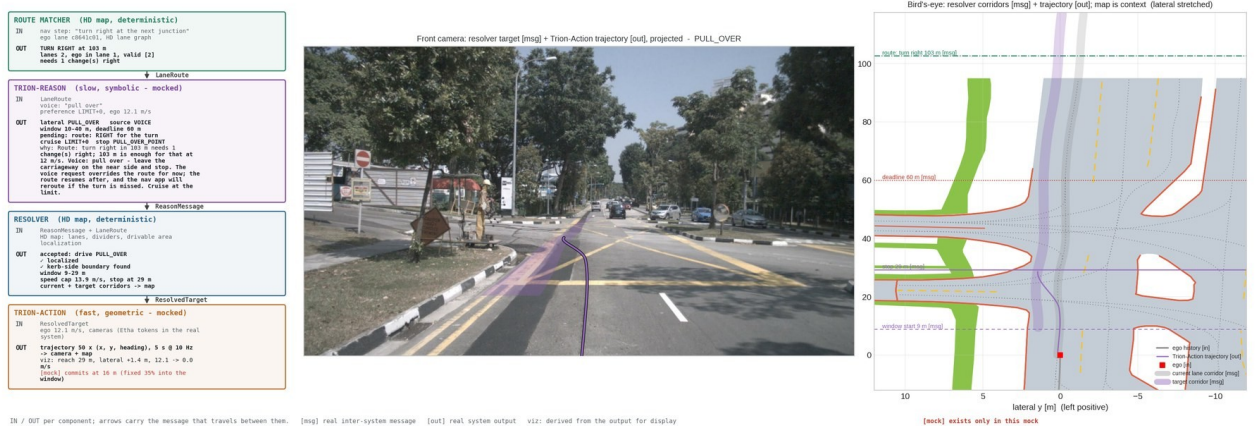


Figure 5. UC4: the override, with the deferred route goal carried as 'pending'.

UC5 · A voice request the map refuses (scene 147 + 'change to the left lane')

Inputs: voice 'change to the left lane'; the ego is already in the kerb lane.

- **Trion-Reason:** emits LEFT - it does not consult lane geometry, and should not need to.
- **Resolver:** the only 'lane' on the left is a map record 0.0 m from the current centreline - an overlapping segment at a split, not a lane → rejected, fallback KEEP, reason shown.
- **Trion-Action:** holds the lane.

Outcome: the interface's safety property in one picture: a bad request produces a logged refusal, not a swerve into the kerb.

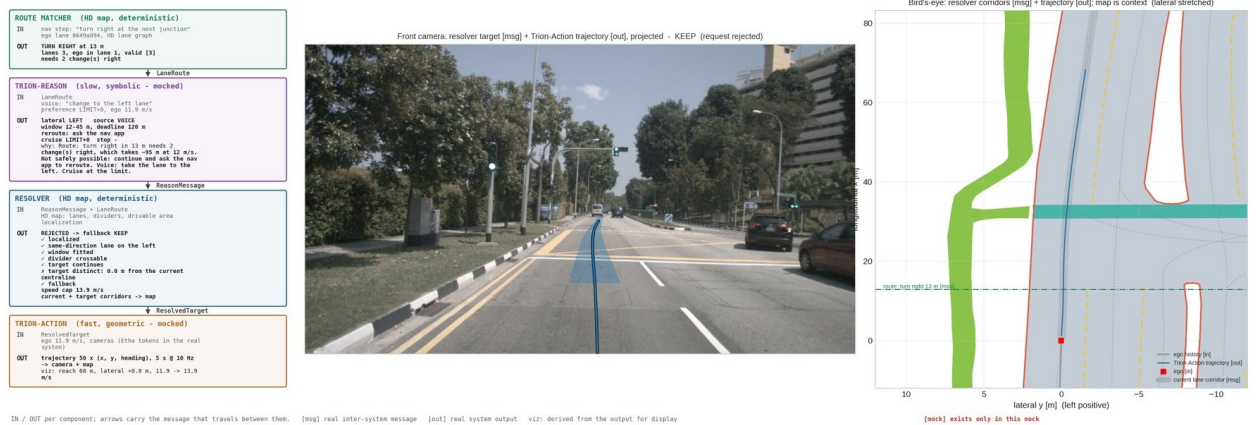


Figure 6. UC5: rejection with the failing check named.

UC6 · Already in the correct lane (scene 147 + 'turn left')

Inputs: nav step 'turn left at the next junction'; ego in the kerb lane.

- **Route Matcher:** left turn at 204 m; the ego's lane is a valid lane; no change needed.
- **Trion-Reason:** KEEP; the turn itself is taken at the junction via the route's connector.

Outcome: most of the time the right answer is to do nothing, and the system says so explicitly.

UC7 · Arrive at the destination (scene 189 + 'arrive on the left')

Inputs: nav step 'arrive: destination on the left'.

- **Route Matcher:** ARRIVE at ~60 m on the left.
- **Trion-Reason:** PULL_OVER from source ROUTE - the same manoeuvre the voice command produces.
- **Resolver / Trion-Action:** as UC4.

Outcome: destination handling reuses the pull-over path rather than needing its own.

UC8 · Interactive scenarios (Trion-Action only; not mocked)

A lead vehicle brakes; a pedestrian steps off the kerb; a light turns amber; a car closes fast in the target lane during a change. None of these reach Trion-Reason. Trion-Action perceives them from the cameras and reacts within its 10 Hz loop: it may reduce speed, delay the commit point, abort a change that has not committed, or brake hard. Each override is reported on Feedback. The design consequence is that Trion-Action needs real perception - including traffic-light state - not just target following; this is the single largest demand the split places on System 1.

UC9 · Map disagrees with the world (not mocked)

Cones close the target lane; localization snaps to the wrong lane; the route turn has no connector in the HD map. The Resolver rejects on missing geometry; Trion-Action reports a map conflict when perceived lanes disagree with the corridor; Trion-Reason falls back to 'follow the road' and, where it can see the cause, says so. These must be first-class states, not exceptions - in practice they are the commonest failures of any map-based stack.

6. How we will know it works

- **Route Matcher and Resolver:** deterministic, so unit-tested against the map. Coverage metrics: fraction of route steps matched; fraction of requests accepted, fitted or rejected, and why.
- **Trion-Reason:** lane-choice agreement with the route-consistent future; pull-over point accuracy; cruise-preference adherence; feasibility judgements versus outcomes. All symbolic, all cheap, derivable from logs.
- **Trion-Action:** closed-loop safety and comfort given ground-truth targets first, then given predicted ones. Open-loop displacement error is not sufficient: it mixes longitudinal and lateral error and hid steering quality in our baseline study.
- **End to end:** closed-loop simulation with the Feedback channel logged, so every failure is attributable to a component.

7. What exists today and what comes next

phase	content	status
0	End-to-end mock: rule-based Trion-Reason and Trion-Action, real Route Matcher and Resolver on the nuScenes HD map, four scenes, interface exercised and visualised	done
1	Productionise Route Matcher and Resolver against the real HD map and nav SDK; add Feedback; unit tests and coverage metrics	next
2	Trion-Action: condition the fast planner on ResolvedTarget instead of a fixed command; closed-loop evaluation with ground-truth targets	after buy-in
3	Trion-Reason: train the reasoning model to emit ReasonMessage from LaneRoute, voice and scene context; labels derived per §6	after buy-in
4	Integrated closed-loop evaluation and driver-facing explanations from the 'why' field	

8. Risks and open questions

- **The interface is deliberately lossy.** A lane goal plus a cruise preference cannot express 'creep to see past the truck' or 'let the merging car in'. By design those are Trion-Action's, but the boundary will be tested in practice.
- **Localization becomes load-bearing.** Snap into the wrong lane and the corridor is confidently 3.5 m off. Trion-Action must keep perceiving lanes and report conflicts.
- **Map staleness.** Construction and temporary markings. The 'follow the road' fallback and the map-conflict flag must be first-class.
- **The commit point is not observable.** A real planner does not announce where it decided to go; it just goes. If reviewers want that number, Trion-Action must be designed to emit it.
- **Traffic lights straddle the boundary.** Which light applies is static (Trion-Reason); reading its state is fast (Trion-Action). Both halves must be built.
- **Speed limit source.** The mock assumes 50 km/h; production needs the limit from the map or the nav SDK, with a road-class fallback.
- **Voice/route conflict policy.** Comply-then-reroute versus refuse-and-explain is a product decision, not a technical one. We propose comply for pull-over and speed, refuse with an explanation for a lane request that contradicts a turn within about 150 m.

9. Decisions requested

1. Adopt the two-system split with the boundary drawn by time constant (§2).
2. Adopt the four messages in §4 as the contract both teams build against.
3. Agree that Trion-Reason handles no interactive scenario and that Trion-Action therefore carries full perception and safety authority (§2.3, UC8).
4. Agree the Resolver and Route Matcher are deterministic, map-backed modules owned separately from either model.
5. Approve Phase 1, and Phases 2 and 3 in parallel once the contract is fixed.

Appendix A · A message trace from the mock (UC2, scene 189)

Abridged from the mock's output; arrays are shown by shape with first and last rows.

```
LaneRoute      {manoeuvre: TURN RIGHT, at_m: 127.1, lanes: 3, current_lane: 0,
                valid: [2], changes_needed: 2, direction: RIGHT, matched:
true}

ReasonMessage  {lateral: RIGHT, window_m: [10.0, 51.1], deadline_m: 70.9,
cruise: 0,
                planned_stop: null, which_light: STRAIGHT, source: ROUTE,
                pending: "+1 RIGHT before the junction", reroute: false,
                why: "Route: turn right in 127 m needs 2 change(s) right; 127
m is
                enough for that at 12 m/s. Cruise at the limit."}

ResolvedTarget {lateral: RIGHT, requested: RIGHT, fallback: false,
                window_m: [20.0, 51.1], deadline_m: 70.9, speed_cap: 13.89,
stop_x: null,
                checks: [localized ✓, same-direction lane on the right ✓
(continuous 20-92 m),
                window fitted ✓ (10-51 → 20-51, junction beside the
road at 8-16 m),
                divider crossable ✓ (DOUBLE_DASHED_WHITE),
                target continues ✓ (167 m), target distinct ✓ (3.4
m)],
                current_corridor: [123, 2], target_corridor: [156, 2]}

Trajectory     shape [50, 3]  first (1.26, 0.09, 0.00) ... last (68.83, -3.73, -
0.01)
```

Appendix B · Assumptions of the mock

- No neural network runs. Trion-Reason and Trion-Action are rule-based stand-ins; Route Matcher and Resolver are real map logic.
- Speed limit assumed 50 km/h; the nuScenes map has none.
- 'Commits at N m' is a fixed 35% into the window, standing in for gap acceptance.
- Lateral movement is reported against the current lane's centreline, so following a curving lane reads as zero and a lane change as one lane width.
- The nuScenes map has no road-edge layer; the drivable-area boundary stands in for one.

Appendix C · Glossary

term	meaning
System 1 / Trion-Action	the fast, geometric, reactive planner (10 Hz)
System 2 / Trion-Reason	the slow, symbolic behaviour planner (~1 Hz)

Resolver	deterministic module turning symbols into validated map corridors
Route Matcher	deterministic module turning a road-level route into lane-level facts
window	the distance range within which a manoeuvre should be completed
deadline	the distance after which a goal is no longer wanted
corridor	a lane centreline with lateral tolerance, map-anchored
scoped override	a voice goal that wins until done, then yields to the route